

Boucles imbriquées, listes et dictionnaires

TME 4

Objectifs

- ★ Boucle
- ★ Chaîne de caractères, Liste et Dictionnaire
- ★ Compréhension de liste et de dictionnaires
- ★ Tests

Rendu

- ⇒ Les réponses seront écrites dans un fichier `nom.n°etu.tp4` où `nom` est votre nom et `n°etu` est votre numéro étudiant.
- ⇒ Si vous avez plusieurs fichiers :
 - Créer un répertoire `rendu TP4_votrenom.n°etu` où `votrenom` est votre nom et `n°etu` est votre numéro étudiant.
 - Copier l'ensemble des fichiers à rendre dans ce répertoire.
 - Archivez le répertoire à l'aide de la commande suivante
`tar czvf tp4_votrenom.n°etu.tgz TP4_votrenom.n°etu`
- ⇒ Charger votre fichier sur Moodle dans la devoir TP4

Exercice 1 Conversion Binaire/Décimal/Héxadécimal

Dans cette exercice, nous n'utiliserons pas les fonctions python `int`, `bin` et `hex`. Nous les codons

Question 1

Écrire une fonction qui prend en paramètre un nombre et un nombre de bits b et retourne une chaîne de caractère de sa représentation binaire sur b bits.

```
>>> decimal_to_binaire(0, 8)
'0b00000000'
>>> decimal_to_binaire(35, 8)
'0b00100011'
>>> decimal_to_binaire(-5, 8)
'0b11111011'
```

Question 2

Écrire une fonction qui prend en paramètre un nombre et un nombre de bits b et retourne une chaîne de caractère de sa représentation hexadécimale sur b bits.

```
>>> decimal_to_hexa(4, 8)
'0x04'
>>> decimal_to_hexa(26, 8)
'0x1A'
>>> decimal_to_hexa(-1, 12)
'0xFFFF'
```

Exercice 2 Triplets numériques (from LU1IN011)

Dans cet exercice, nous expérimentons les compréhensions multiples en manipulant des listes de triplets d'entiers.

Question 1

Donner une définition de la fonction triplets qui retourne la liste des triplets (i, j, k) sur l'intervalle $[1; n]$ (avec n un entier naturel).
Par exemple :

```
>>> triplets(0)
[]
>>> triplets(1)
[(1, 1, 1)]
>>> triplets(2)
[(1, 1, 1),
 (1, 1, 2),
 (1, 2, 1),
 (1, 2, 2),
 (2, 1, 1),
 (2, 1, 2),
 (2, 2, 1),
 (2, 2, 2)]
```

Remarque : votre fonction utilisera une compréhension

Question 2

On souhaite maintenant lister les décompositions sur un intervalle $[1; n]$, c'est-à-dire les triplets (i, j, k) tels que $i + j = k$.

Par exemple :

```
>>> decompositions(0)
[]
>>> decompositions(1)
[]
>>> decompositions(2)
[(1, 1, 2)]
>>> decompositions(3)
[(1, 1, 2), (1, 2, 3), (2, 1, 3)]
>>> decompositions(4)
[(1, 1, 2), (1, 2, 3), (1, 3, 4), (2, 1, 3), (2, 2, 4), (3, 1, 4)]
```

Question 3

On souhaite maintenant lister les encadrements sur un intervalle $[1; n]$, c'est-à-dire les triplets (i, j, k) tels que $i \leq j \leq k$.

Par exemple :

```
>>> encadrements(0)
[]
>>> encadrements(1)
[(1, 1, 1)]
>>> encadrements(2)
[(1, 1, 1), (1, 1, 2), (1, 2, 2), (2, 2, 2)]
>>> encadrements(4)
[(1, 1, 1),
 (1, 1, 2),
 (1, 1, 3),
 (1, 1, 4),
 (1, 2, 2),
 (1, 2, 3),
 (1, 2, 4),
 (1, 3, 3),
 (1, 3, 4),
 (1, 4, 4),
 (2, 2, 2),
 (2, 2, 3),
 (2, 2, 4),
 (2, 3, 3),
 (2, 3, 4),
 (2, 4, 4),
 (3, 3, 3),
 (3, 3, 4),
 (3, 4, 4),
 (4, 4, 4)]
```

Exercice 3 Compter les mots

Question 1

Écrivez une fonction qui prend un nom de fichier texte en paramètre, charge son contenu, découpe le texte en mots et compte, pour chaque mot, le nombre de fois que ce mot apparaît dans le texte.

Pour simplifier, on considèrera qu'un mot est toute séquence de caractères séparée des autres par un espace.

Les tests pourront se faire sur le texte suivant *miserables-chap1-tokenized.txt*.

Question 2

Écrire une fonction qui trouve le mot le plus utilisé dans un fichier texte passer en paramètre. Ne considérez pas les virgules ni les points.

Exercice 4 We are family

Soit une liste de personne de la forme suivante (dans le fichier `persons.py`) :

```
[
    {
        'gender' : 'male',
        'name' : 'Hans',
        'parents' : ['Paul', 'Peter']
    },
    {
        'gender' : 'female',
        'name' : 'Irma',
        'parents' : ['Gertrud', 'Jaques']
    },
    ...
]
```

Question 1

Écrire une fonction qui prend une liste de personnes en paramètre et un nom et retourne la liste des enfants pour le nom donné.

Question 2

Écrire une fonction qui crée un dictionnaire qui pour chaque nom de personne d'une liste associe la liste de ces enfants. Dans l'exemple de la liste précédente, la fonction retournera :

```
{ 'Hans' : ['Sam'],
  'Irma' : [],
  'Lison' : [],
  'Fred' : [],
  'Sam' : ['Lison', 'Fred'] }
```

Exercice 5 L'outil ultime du cruciverbiste (from Xavier Tannier)

Le fichier *liste-mots.txt* (encodé en UTF-8) contient 410441 mots de la langue française (pas de noms propres).

Écrivez une fonction qui permet à un utilisateur qui fait des mots croisés de saisir trois paramètres :

1. la longueur du mot qu'il cherche
2. une lettre dont il sait qu'elle est dans le mot
3. la position à laquelle cette lettre se trouve (l'index de la lettre dans le mot)

La fonction retournera l'ensemble des mots répondant à ces critères.

Il vous est recommandé de passer par la construction d'un dictionnaire.

Vous pourrez vérifier que le nombre de mots de 3 lettres ayant un 'a' en dernière position est de 36, le nombre de mots de 4 lettres qui commencent par 'w' est 5 et qu'il n'existe pas de mot de 13 lettres terminant par 'b'.